Prompts for Building Enterprise Applications

Claude Code Prompts - Quick Start Guide

This page provides ready-to-use Claude Code prompts that help a new developer quickly spin up a secure, Sleep Number ready application. By guiding them through project setup, UI design, technology selection, architecture planning, infrastructure as code, multi-tenant design, authentication/authorization, and AI prompt management, it ensures their code aligns with modern security practices and M.A.C.H.-based architectural standards from day one.

These prompts help a new developer quickly design and build a secure, Sleep Number–ready application using Claude Code. It streamlines every stage of setup—from choosing the tech stack and defining architecture to implementing cloud infrastructure, security, multi-tenancy, authentication, and AI prompt management—so that new solutions are both fast to develop and aligned with Sleep Number's security and M.A.C.H.-based architecture standards.

Quick Reference

What do you need?	Use Prompt
Start brand new project	#1
Design pages and UI	#2
Choose technologies	#3
Plan system design	#4
Set up cloud services	#5
Add Docker containers	#6
Support multiple tenants	#7
Add login system	#8
Add user roles	#9
Add Prompt Architecture	#10
Add security	#11

Tips for Non-Developers

Before you start:

1. Know what your app should do (the features)
2. Know who will use it (the users)
3. Know any special requirements (security, compliance, scale)

How to use these prompts:

1. Copy a prompt above
2. Replace all [BRACKETS] with your information
3. Paste into Claude Code

4. Review what Claude creates
5. Ask follow-up questions if needed

Example:
Instead of: `[YOUR PROJECT NAME]`
Write: `Sales Training Platform`

Instead of: `[Feature 1]`
Write: `Video role-play with AI coach`

---

1. Start a New Project

This New Project prompt helps a new developer quickly scaffold a complete, Sleep Number–ready SaaS application from scratch. By standardizing the initial setup (tech stack, structure, and documentation) around approved patterns like Next.js, Azure Functions, and Terraform, it accelerates onboarding, reduces guesswork, and ensures new projects start aligned with Sleep Number's security and M.A.C.H.-based architecture standards.

```
Create a new SaaS project called [YOUR PROJECT NAME].

Purpose: [What does it do?]
Main features: [List 3-5 features]
Users: [Who will use it?]

Set up the project structure with:
- Frontend [Next.js]
- Backend [Azure Functions]
- Infrastructure [Terraform]
- Documentation

```

---

2. Design the User Interface

This Design the User Interface prompt helps a new developer quickly define all key screens, layouts, and interactions for their application. By standardizing page types (login, dashboard, admin, profile, and custom pages) and emphasizing responsive, user-friendly design, it accelerates UI planning, reduces rework, and ensures the interface is intuitive and consistent across mobile and browser experiences.

```
Design a modern web interface for [YOUR PROJECT NAME].

Pages needed:
- Login page
- Dashboard (main page after login)
- [Your page 1]
- [Your page 2]
- [Your page 3]
- Admin panel
- User profile

Make it responsive and easy to use on mobile and broswer.
```

---

3. Choose Technology Stack

This Choose Technology Stack prompt helps a new developer quickly identify the right tools and platforms for their application based on real requirements like scale, security, and special features. By steering them toward Sleep Number–aligned patterns (Next.js, Azure Functions, PostgreSQL, Azure services), it reduces trial-and-error, ensures technical decisions support compliance and performance needs, and keeps new projects consistent with established architecture standards.

```
Recommend the best technology stack for [YOUR PROJECT NAME].
```

```

Requirements:
- Type: [Web app / Mobile app / Both]
- Users: [How many users expected?]
- Security: [Any compliance needs like HIPAA, SOC2?]
- Special features: [AI, real-time chat, file uploads, etc.]

Suggest similar technologies to PULSE:
- Next.js frontend
- Azure Functions backend
- PostgreSQL database
- Azure cloud services
```

---

4. Design the Architecture

This Design the Architecture prompt helps a new developer map out a complete, M.A.C.H.-aligned system design for their project. By defining service boundaries, required APIs, frontend–backend interactions, and recommended Azure cloud services around key features, it reduces architectural guesswork, supports security and scalability from the start, and keeps new solutions consistent with Sleep Number's reference architecture.

```
Design the system architecture for [YOUR PROJECT NAME] using M.A.C.H. principles.

Key features:
- [Feature 1]
- [Feature 2]
- [Feature 3]

Show me:
- How services should be separated
- What APIs are needed
- How frontend and backend connect
- Cloud services to use
```

---

5. Create Cloud Infrastructure

This Create Cloud Infrastructure prompt helps a new developer generate Terraform-based Cloud resources for their project in a consistent, secure way. By standardizing how web apps, functions, databases, storage, and AI services are provisioned—with private networking, secret management, and encryption built in—it accelerates environment setup while ensuring cloud infrastructure aligns with Sleep Number's security and architecture standards.

```
Create Terraform configuration for [YOUR PROJECT NAME] on Azure.

Services needed:
- Web app for frontend
- Functions for backend
- [PostgreSQL / MongoDB / other] database
- File storage
- [AI services like OpenAI - if needed]

Security requirements:
- Private networking (no public access)
- All secrets in Azure Key Vault
- Encryption everywhere
```

---

6. Add Docker Support

This Add Docker Support prompt helps a new developer containerize their application's frontend and backend in a consistent, production-ready way. By generating secure Docker images and a docker-compose setup for local development, it simplifies environment management, reduces "it works on my machine" issues, and makes it easier to deploy Sleep Number–ready services across different environments.

```
Create Docker containers for [YOUR PROJECT NAME].

Components to containerize:
```

```
- [Frontend: Next.js / React / other]
- [Backend: Python / Node.js / other]

Make containers secure and optimized for production.
Include docker-compose.yml for local development.
```

---

7. Add Multi-Tenant Support (if applicable)

This Add Multi-Tenant Support prompt helps a new developer design their application to securely serve multiple organizations (i.e., Wellness, Corporate and Digital) from a shared platform. By standardizing tenant isolation (separate users and data), optional branding, and subscription tiers on a shared database model, it reduces complexity, prevents cross-tenant data leaks, and ensures new solutions are built with scalable, enterprise-ready multi-tenant architecture from the start.

```
Make [YOUR PROJECT NAME] support multiple organizations (multi-tenant).

Tenant type: [B2B companies / B2C users / Both]

Each tenant should have:
- Their own users
- Separate data
- Custom branding [optional]
- Different subscription levels [if needed]

Use shared database with tenant_id column
```

---

8. Add User Login

This Add User Login (OIDC) prompt helps a new developer integrate secure, standards-based single sign-on into their application using providers like Entra ID, Okta, or Google Workspace with NextAuth.js. By standardizing authentication around OpenID Connect, it reduces custom auth complexity, improves user experience with SSO, and ensures login flows align with modern security and identity management practices.

```
Add user authentication to [YOUR PROJECT NAME] using Open Identity Connect (OIDC) and NextAuth.js.

Login provider: [Entra ID / Okta / Google Workspace]
Session length: [8 hours / 24 hours / other]

Users should:
- Sign in with SSO (single sign-on)
- Stay logged in securely
- Automatically be created on first login
```

---

9. Add User Permissions

This Add User Permissions prompt helps a new developer design a clear, role-based access control (RBAC) model for their application. By defining standard roles and mapping each one to specific permissions, it reduces ad-hoc security decisions, prevents unauthorized access to sensitive features, and ensures the app's authorization layer is consistent, auditable, and aligned with enterprise security expectations.

```
Create a RBAC Authorization permission system for [YOUR PROJECT NAME].

User roles needed:
- [Admin: Full access]
- [Manager: Team access]
- [User: Basic access]
- [Your other roles]

Permissions:
- [Who can access admin panel?]
- [Who can manage users?]
- [Who can view reports?]
```

```
- [Your other permissions]
```

---

10. Design AI Prompt Architecture (if applicable)

This Fully Manageable In-App AI Prompt Architectures prompt helps a developer design an application where all AI prompts are centrally managed through the UI instead of being buried in code. By making prompts discoverable, editable, and testable inside the app, it removes the need to search source files, speeds up iteration and refinement, and empowers product and business users to safely tune AI behavior without requiring rework of the entire application.

```
Design the AI prompt management system for [YOUR PROJECT NAME] where
all AI prompts are centrally managed through the UI By making prompts
discoverable, editable, and testable inside the app.

Admin features needed:
- View all active prompts
- Edit prompts without code changes
- Test prompts before deploying
- Compare prompt versions
- Rollback to previous versions
- See which prompts are used where

Storage:
- Store prompts in [database / file system]
- Keep history of all changes
- Track who changed what and when

User interface:
- Simple editor for non-technical users
- Preview how prompts will look to AI
- Test prompt with sample inputs
- Approve changes before going live
```

---

11. Secure Everything

This Secure Everything prompt helps a new developer apply Zero Trust security principles across their entire application and cloud environment. By standardizing protections for networking, data encryption, secrets, input validation, AI safety, and access control, it reduces security gaps, supports compliance needs, and ensures new solutions are built with strong, end-to-end safeguards from the start.

```
Implement security for [YOUR PROJECT NAME] following Zero Trust
principles.

Protect:
- Network: Use private connections for all services
- Data: Encrypt everything in transit and at rest
- Secrets: Store all passwords/keys in Azure Key Vault
- Input: Validate all user input
- [AI: Protect against prompt injection - if using AI]
- Access: Only authenticated users with right permissions

Rate limits: [Requests per hour per user, if needed]
```